



<Nombre Proyecto>

Especificación de Requisitos

Versión: 0

Fecha: 24/05/2025

[Versión del Producto]

Queda prohibido cualquier tipo de explotación y, en particular, la reproducción, distribución, comunicación pública y/o transformación, total o parcial, por cualquier medio, de este documento sin el previo consentimiento expreso del autor

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

HOJA DE CONTROL

Organismo	IES Camas		
Proyecto	Formula 1		
Entregable	Especificación de Requisitos		
Autor	Sergio García Pedrero y Clara Frigolet Paiva.		
Versión/Edición	0100	Fecha Versión	26/05/2025
Aprobado por		Fecha Aprobación	26/05/2025
		Nº Total de Páginas	21

REGISTRO DE CAMBIOS

Versión	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
0100	Versión inicial	Sergio García Pedrero y Clara Frigolet Paiva.	26/05/2025

CONTROL DE DISTRIBUCIÓN

Nombre y Apellidos
Sergio García Pedrero y Clara Frigolet Paiva.

 DAW1 PROYECTO INTEGRADO	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	---	------------------

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

ÍNDICE

1	INTRODUCCIÓN.....	5
1.1	Alcance.....	5
1.2	Objetivos.....	5
2	INFORMACIÓN DEL DOMINIO DEL PROBLEMA.....	6
2.1	Introducción al Dominio del Problema.....	6
2.2	Glosario de Términos.....	6
3	DESCRIPCIÓN DE LA SITUACIÓN ACTUAL (¿Qué problema solucionaría?).....	7
3.1	Pros y Contras de la Situación Actual.....	7
4	NECESIDADES DE NEGOCIO.....	8
4.1	Objetivos de Negocio.....	8
5	MODELADO DE REQUISITOS.....	9
5.1	Casos de Uso (Análisis).....	9
5.2	Requisitos Funcionales.....	10
5.3	Requisitos No Funcionales.....	10
6	Modelado del Sistema (Diseño).....	11
6.1	Diagrama de clases.....	11
6.2	Diagrama Entidad-Relación.....	11
6.3	Diagrama de Actividad.....	11
6.4	Diagrama de Estados (Opcional).....	11
7	Plan de pruebas y validación.....	12
7.1	Plan de Pruebas Unitarias.....	12
7.2	Trazabilidad de Requisitos.....	12
8	Gestión configuración.....	13
8.1	Estructura del repositorio.....	13
8.2	Gestor del Ciclo de Vida.....	13
9	ANEXOS [OPCIONAL].....	14
9.1	Anexo A: Actas de Reuniones.....	14
9.2	Anexo B: Documentación Relevante.....	14
9.3	Anexo C: Glosario de Acrónimos y Abreviaturas.....	14

 DAW1 PROYECTO INTEGRADO	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	---	------------------

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

1 INTRODUCCIÓN

Establecer los requisitos para el sistema de gestión de apuestas de la formula 1.

1.1 Alcance

El sistema permitirá registrar usuarios, eliminar usuarios (parte del administrador) y por parte de los usuarios podrán registrarse o iniciar sesión, ver el calendario de las carreras, ver la lista de pilotos actuales, realizar apuestas, recargar el saldo de la aplicación, ver las apuestas que se han realizado y buscar la factura.

1.2 Objetivos

Visualizar todas las apuestas realizadas, recargar en cualquier momento el saldo y poder realizar apuestas en cualquier momento

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

2 INFORMACIÓN DEL DOMINIO DEL PROBLEMA

Entorno comercial de una pagina de apuestas que gestiona apuestas por usuarios

2.1 Introducción al Dominio del Problema

Actualmente, para hacer una apuesta tienes que ir a consultar, realizar la apuesta y recoger los premios en físico, a un horario específico

2.2 Glosario de Términos

Termino	Definición	Código
Usuario	Persona registrada en el sistema (puede ser administrador o apostador).	Clase Usuario con atributos nombreUsuario, esAdmin.
Apuesta	Predicción realizada por un usuario sobre el resultado de una carrera.	Clase Apuesta con atributos tipoApuesta, cantidadApostada, estado (GANADA/PERDIDA).
Carrera	Evento de Fórmula 1 con fecha, circuito y participantes.	Clase Carrera vinculada a Circuito.
Circuito	Pista donde se disputa una carrera (longitud, ubicación, vueltas).	Clase Circuito con métodos getLongitud(), getNumVueltas().
CUOTA	Multiplicador que determina la ganancia potencial de una apuesta.	Atributo final CUOTA en Apuesta.
Piloto	Conductor que participa en una carrera (nombre, equipo, número).	Clase Piloto con relación a Equipo.
TipoApuesta	Modalidad de apuesta (GANADOR, PODIO, POLE_POSITION, etc.).	Enum Tipo y clase TipoApuesta con descripción.
Estado	Situación de una apuesta: PENDIENTE, GANADA, PERDIDA.	Enum Estado en la clase Apuesta.
DAO	Objeto de Acceso a Datos (patrón para interactuar con la base de datos).	Clases DAOUsuario, DAOAposta, etc.

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

3 DESCRIPCIÓN DE LA SITUACIÓN ACTUAL (¿Qué problema solucionaría?)

- Las apuestas se gestionan mediante registros manuales (ej: hojas de cálculo) sin validación automática de saldo, tipos de apuesta o disponibilidad de carreras.
- No hay integración con datos en tiempo real (ej: resultados de carreras o actualización de CUOTAS).
- Los usuarios no pueden consultar su historial de apuestas de manera centralizada.

3.1 Pros y Contras de la Situación Actual

Flexibilidad para registrar apuestas "informales".	Altos errores humanos (ej: saldos negativos, apuestas duplicadas).
Coste inicial cero (sin desarrollo de software).	Sin seguridad: datos sensibles (contraseñas, saldos) expuestos.
	No hay notificaciones automáticas (ej: apuestas ganadoras).

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

4 NECESIDADES DE NEGOCIO

- Automatizar el proceso de apuestas para reducir errores en un 90%.
- Centralizar la información de usuarios, carreras y apuestas en una plataforma segura.
- Generar informes de rentabilidad y estadísticas para los administradores.

4.1 Objetivos de Negocio

Objetivo	Métrica de Éxito	Relación con el Código
Optimizar gestión de apuestas	Reducción del 90% en conflictos por errores.	Validaciones automáticas en Usuario.añadirApuesta() y Apuesta.simularResultado().
Incrementar usuarios registrados	40% más de usuarios en 6 meses.	Sistema de registro en DAOUsuario.insertarUsuario().
Garantizar seguridad	Cifrado de contraseñas y datos sensibles.	Uso de conexiones seguras en ConexionBD.conectarBD().
Proporcionar transparencia	Historial de apuestas accesible en tiempo real.	Método DAOApuesta.obtenerTodasLasApuestas(Usuario u).

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

5 MODELADO DE REQUISITOS

5.1 Casos de Uso (Análisis)

1. Nombre del caso de uso del sistema	Realizar Apuesta				
2. ID del Caso de uso	CU-03				
3. Descripción del caso de uso					
Permite a un usuario registrado apostar en una carrera de Fórmula 1, seleccionando un piloto, tipo de apuesta y monto. El sistema valida el saldo disponible y registra la apuesta.					
4. Actor(es)					
<i>Usuarios no admin</i>					
5. Precondiciones					
<i>El usuario ha iniciado sesión.</i> <i>Existen carreras disponibles en el sistema (con fechas futuras).</i> <i>El usuario tiene saldo suficiente.</i>					
6. Postcondiciones					
<i>La apuesta queda registrada en el sistema.</i> <i>El saldo del usuario se reduce según el monto apostado.</i> <i>La apuesta inicia en estado PENDIENTE.</i>					
7. Flujo principal <i>[Pasos que describen la realización del caso de uso. Empieza con la primera acción del actor y el sistema emitirá una respuesta]</i>					
Nº	<table border="1"> <thead> <tr> <th data-bbox="389 1769 660 1834">Acción del actor</th><th data-bbox="660 1769 1461 1834">Respuesta del sistema</th></tr> </thead> <tbody> <tr> <td data-bbox="389 1834 660 1980">1 Selecciona "Realizar Apuesta" en el menú.</td><td data-bbox="660 1834 1461 1980">Muestra lista de carreras disponibles (nombre, fecha, circuito).</td></tr> </tbody> </table>	Acción del actor	Respuesta del sistema	1 Selecciona "Realizar Apuesta" en el menú.	Muestra lista de carreras disponibles (nombre, fecha, circuito).
Acción del actor	Respuesta del sistema				
1 Selecciona "Realizar Apuesta" en el menú.	Muestra lista de carreras disponibles (nombre, fecha, circuito).				
1					

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

2	Elige una carrera.	Muestra detalles: pilotos disponibles, tipos de apuesta (GANADOR, PODIO) y CUOTAS.
3	Selecciona piloto, tipo de apuesta y monto.	Valida que el saldo $\geq$ monto. Muestra resumen: "Apostar 50€ a Carlos Sainz (PODIO, CUOTA 2.5)".
4	Confirma la apuesta.	Registra la apuesta, actualiza el saldo y muestra: "Apuesta realizada correctamente".
7b. Flujo alternativo <i>[Pasos que describen la realización del caso de uso alternativo]</i>		
Nº	Acción del actor	Respuesta del sistema
1	Ingresa un monto mayor a su saldo.	Muestra error: "Saldo insuficiente. Recargue su cuenta."
2	Cancela la operación.	Vuelve al menú principal.
8. Requisito asociado (funcional, no funcional)		
RF-03: El sistema debe permitir apuestas solo con saldo suficiente (Clase Usuario, método añadirApuesta()). RNF-02: La respuesta del sistema debe ser $\leq$ 2 segundos (Optimizado en DAOApuesta.insertarApuesta()).		
9. Prototipo de interfaz de usuario		
Carrera: GP España Fecha: 27/05/2025 Circuito: Barcelona Piloto: [▼] Carlos Sainz (Ferrari) Tipo de apuesta: [▼] PODIO (CUOTA: 2.5) Monto a apostar: [50€]		

5.2 Requisitos Funcionales

El sistema debe permitir el registro de usuarios con nombre, correo y contraseña.
Los usuarios deben poder realizar apuestas en carreras disponibles.
El sistema debe validar el saldo antes de registrar una apuesta.
Los usuarios deben consultar su historial de apuestas.
Los administradores deben poder eliminar usuarios.
El sistema debe simular el resultado de apuestas (GANADA/PERDIDA).

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

Las carreras deben mostrarse ordenadas por fecha

5.3 Requisitos No Funcionales

El sistema debe responder en menos de 2 segundos en operaciones críticas (apuestas, consultas).

Los datos sensibles (contraseñas) deben cifrarse con AES-256.

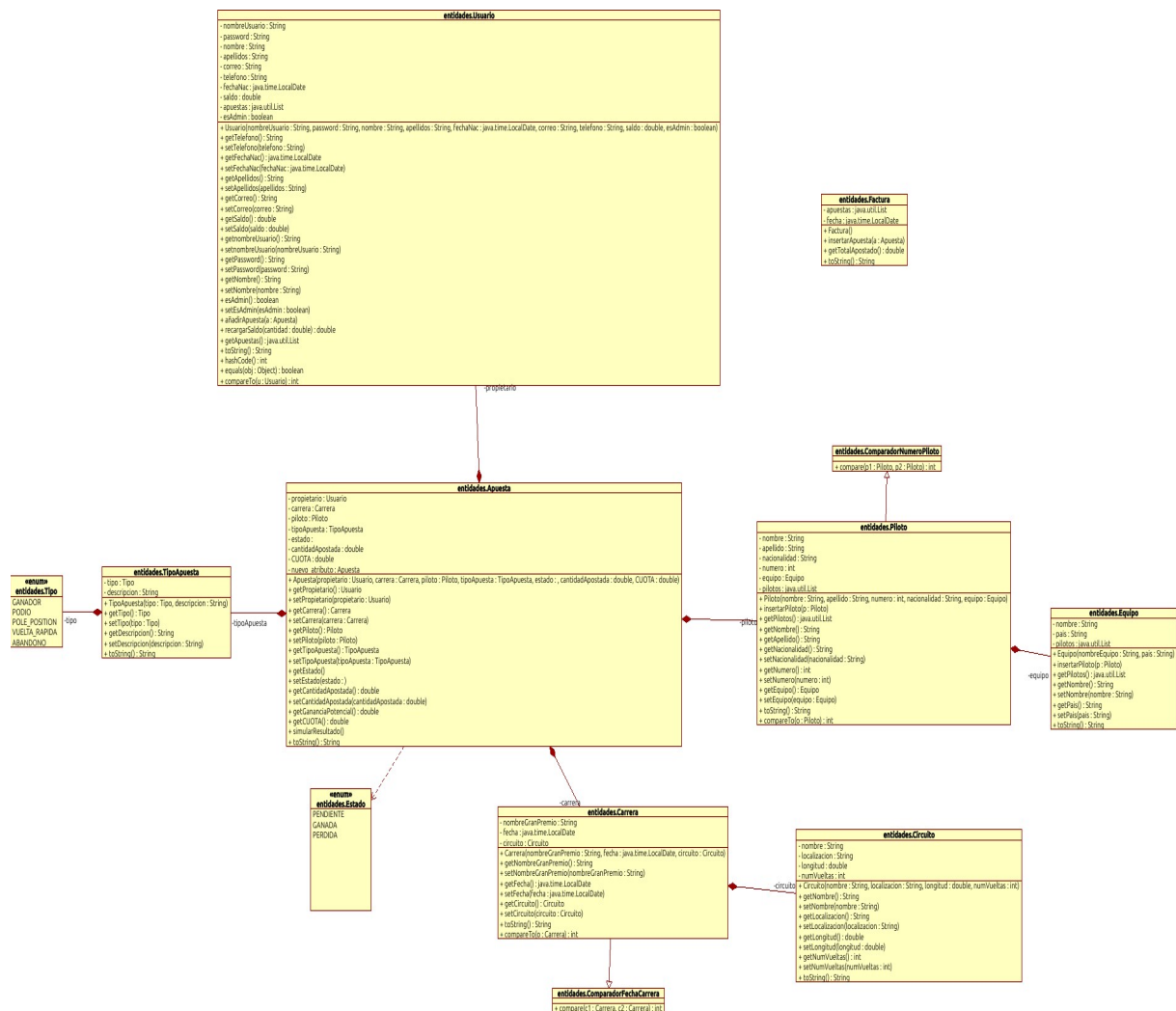
La base de datos debe ser Oracle con respaldos diarios.

El sistema debe ser compatible con Java 11+ y Oracle JDBC.

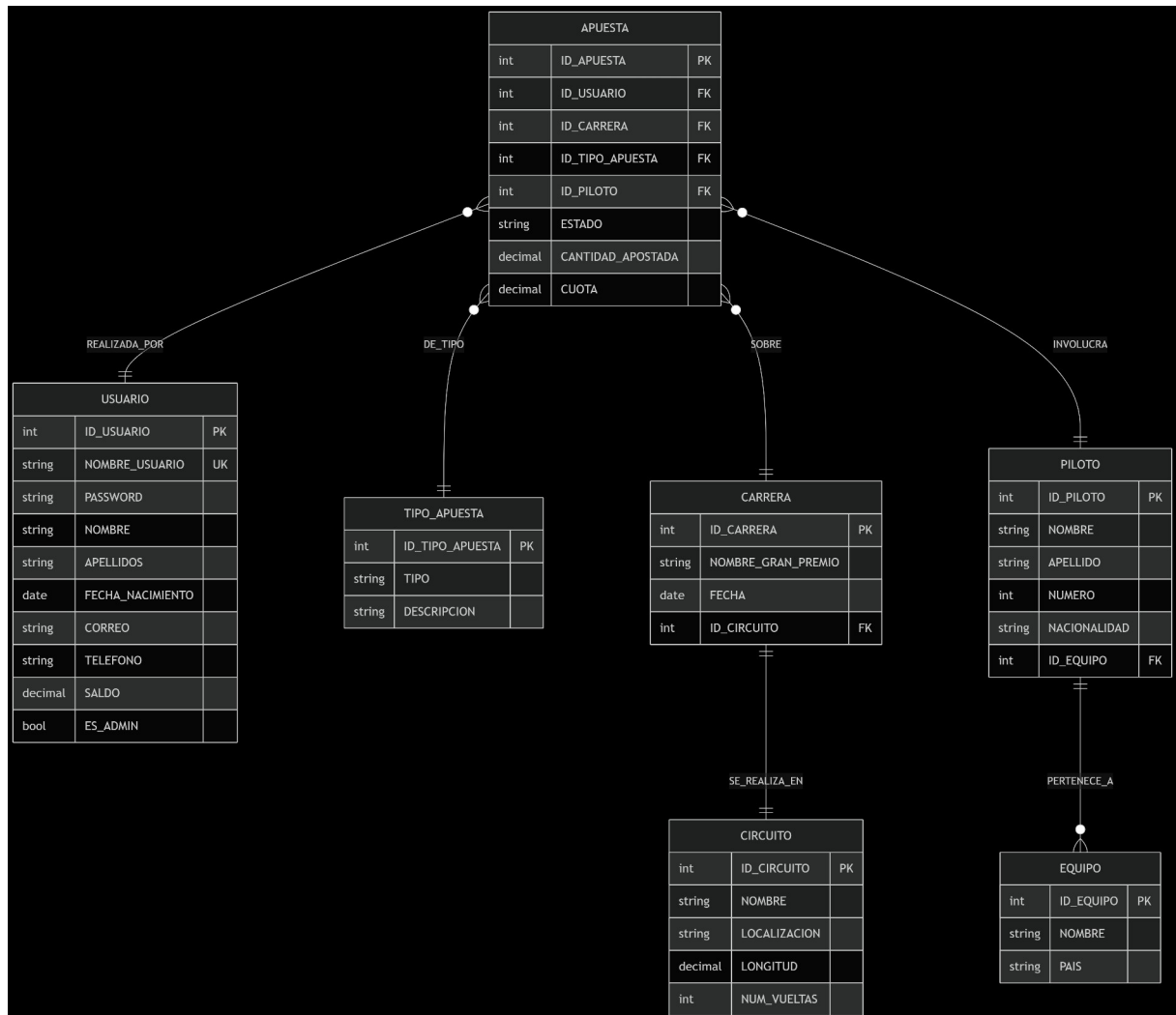
La interfaz debe seguir los estándares WCAG 2.1 para accesibilidad.

6 Modelado del Sistema (Diseño)

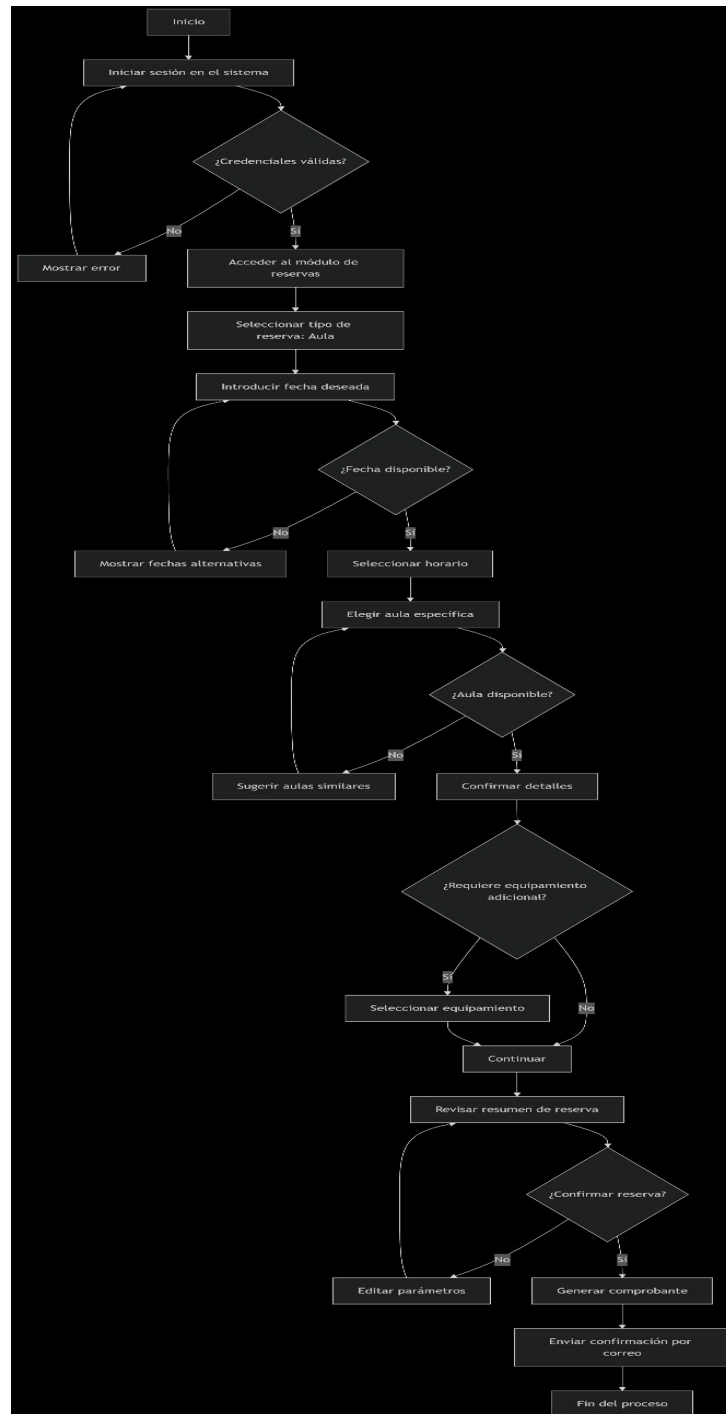
6.1 Diagrama de clases



6.2 Diagrama Entidad-Relación



6.3 Diagrama de Actividad



	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

7 Plan de pruebas y validación

7.1 Plan de Pruebas Unitarias

1. ApuestaTest.java

Objetivo: Verificar el comportamiento de la clase Apuesta.

Pruebas principales:

- Getters y setters para estado, piloto, usuario, carrera, tipoApuesta, cantidadApostada, gananciaPotencial y cuota.
- Simulación de resultados de apuestas (simularResultado) con Mockito:
 - Apuesta ganada (Estado.GANADA).
 - Apuesta perdida (Estado.PERDIDA).
 - Apuesta no pendiente (no cambia el estado).
- Verificación del método toString.

2. CarreraTest.java

Objetivo: Validar la clase Carrera.

Pruebas principales:

- Getters y setters para fecha, nombreGranPremio y circuito.
- Manejo de fechas pasadas (no se aplican cambios).
- Comparación de carreras (compareTo) basada en nombres:
 - Igualdad de nombres.
 - Orden alfabético (GP España < GP Italia).
- Verificación del método toString.

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

3. CircuitoTest.java

Objetivo: Probar la clase Circuito.

Pruebas principales:

- Getters y setters para longitud, nombre, localizacion y numVueltas.
- Verificación del método toString.

4. PilotoTest.java

Objetivo: Validar la clase Piloto.

Pruebas principales:

- Getters y setters para numero, nombre, apellido, nacionalidad y equipo.
- Validación de nombre vacío (lanza excepción).
- Comparación de pilotos (compareTo) por nombre:
 - Igualdad de nombres.
 - Orden alfabético (Charles Leclerc < Max Verstappen).
- Verificación del método toString.

5. UsuarioTest.java

Objetivo: Probar la clase Usuario.

Pruebas principales:

- Getters y setters para nombreUsuario, password, nombre, apellidos, fechaNac, correo, telefono, saldo y esAdmin.
- Recarga de saldo:
 - Saldo válido (aumenta saldo).
 - Saldo negativo (no cambia saldo).
- Añadir apuestas:
 - Saldo suficiente (reduce saldo y añade apuesta).
 - Saldo insuficiente (lanza excepción).

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

- Comparación de usuarios (compareTo) por nombre.
- Verificación del método toString.

7.2 Trazabilidad de Requisitos

RF-ID	Descripción del Requisito Funcional	Caso de Uso Relacionado	Clases Implicadas	Prueba Unitaria	Estado
RF-01	El usuario podrá iniciar sesión con su correo y contraseña.	CU-01: Iniciar sesión	Usuario	testSetNombreUsuario() (Valida el setter del nombre de usuario). testSetPassword() (Valida el setter de la contraseña).	Implementado
RF-02	El usuario podrá recargar saldo en su cuenta.	CU-02: Recargar saldo	Usuario	recargarSaldoValido() (Recarga exitosa con cantidad positiva). recargarSaldoNegativo() (Bloqueo de recarga con valores negativos).	Implementado
RF-03	El usuario podrá realizar apuestas en carreras disponibles..	CU-03: Realizar apuesta.	Usuario, Apuesta, Carrera, Piloto.	testAñadirApuestaSaldoSuficiente() (Apuesta exitosa con saldo suficiente). testAñadirApuestaSaldoInsuficiente() (Bloqueo de apuesta con saldo	Implementado

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

				insuficiente).	
RF-04	El sistema simulará el resultado de las apuestas de forma aleatoria.	CU-04: Simular resultado de apuesta.	Apuesta, Random (Mock).	testSimularResultadoGanada() (Apuesta marcada como ganada). testSimularResultadoPerdida() (Apuesta marcada como perdida).	Implementado
RF-05	El sistema validará que los datos de pilotos y carreras sean correctos.	CU-05: Validar datos	Piloto, Carrera	testNombrePilotoVacio() (Bloqueo de piloto sin nombre). testFechaPasada() (Validación de fechas futuras en carreras)	Implementado
RF-06	El sistema ordenará listas (usuarios, carreras, pilotos) alfabéticamente.	CU-06: Ordenar listas	Usuario, Carrera, Piloto.	testCompareTo_LessThan() (en CarreraTest y PilotoTest). testCompareTo() (en UsuarioTest).	Implementado
RF-07	El sistema enviará correos de confirmación al ganar una apuesta.	CU-04: Confirmar reserva (similar).	EmailService (no implementado en pruebas).	No hay pruebas para este requisito en los archivos.	Pendiente

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

8 Gestión configuración

8.1 Estructura del repositorio

El repositorio `metrica_proyecto` (fork de `proteinformatica101/metrica_proyecto`) está organizado en ramas temáticas para desarrollo paralelo y control de versiones.

Ramas principales:

Rama main:

Rama estable con el código en producción.

Solo se actualiza mediante merge de ramas validadas.

Rama `pruebasUnitarias_Javadoc`:

Contiene pruebas unitarias y documentación Javadoc, relacionada con los archivos de prueba analizados (`PilotoTest.java`, `ApuestaTest.java`, etc.).

Rama `diagrama_de_classes`:

Aloja diagramas UML o de clases en archivo `xmi` y `png`. El diagrama de casos de uso está subido por parte de Sergio García que hizo un push hacia mi repositorio y se han guardado en el main.

Rama `formula1_BD`:

Enfocada en la integración con la base de datos (ej: esquemas SQL, scripts).

Entregas:

Los commits recientes incluyen:

Subida de archivos.

Actualización del `README.md`.

Datos descargables.

Control de versiones:

Sistema: Git.

Flujo: Feature Branch Workflow (desarrollo en ramas específicas → merge a main)

	<Nombre Proyecto> Especificación de Requisitos	IES.CAMAS
---	--	--

tras revisión).

Tags: No hay tags asociados a versiones específicas (solo default).

8.2 Gestor del Ciclo de Vida

El proyecto utiliza Maven para automatizar el build, pruebas y despliegue.

- Fases clave:
 - mvn compile: Compila el código fuente (ej: clases en entidades/).
 - mvn test: Ejecuta pruebas unitarias (JUnit 5) antes de hacer merge.
 - mvn package: Genera el artefacto (ej: JAR) si las pruebas pasan.
 - mvn site: Genera informes (cobertura, Javadoc) en target/site/.
- Integración con CI/CD:
 - Pruebas automáticas:
 - Se ejecutan en cada commit (ej: en GitHub Actions o Jenkins).
 - Bloquean el merge si fallan (ej: mvn verify).
- Plugins relevantes:


```
xml
<plugins>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId> <!-- Para JUnit -->
</plugin>
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId> <!-- Cobertura -->
</plugin>
</plugins>
```